VEEVA QA • PHASE 1

Software Testing Fundamentals

Complete Phase 1 Learning Guide • Concepts, examples, workflows and revision

- *Build a strong base before learning Selenium or UI automation.*
- *Use one running example to connect requirements, test cases, execution, defects and regression.*
- *Includes SDLC/STLC, testing types and levels, Agile/Scrum, Jira, Xray, Confluence and a hands-on exercise.*

Study method: understand the concept → explain it in your own words → apply it to a requirement → practise interview distinctions.

How to use this guide

Do not passively read this like a story. For each topic, close the PDF and explain the definition, give an example, and state how it differs from a related concept. The goal is to reason like a tester, not recite words.

Learning map

```
REQUIREMENTS → SDLC/STLC → TEST DESIGN → TEST EXECUTION
      → DEFECT MANAGEMENT → TEST LEVELS/TYPES
      → AGILE → JIRA + XRAY + CONFLUENCE
      → MINI PROJECT → REVISION
```

Contents

1. Software testing fundamentals
2. QA, QC, verification and validation
3. SDLC and STLC
4. Test artifacts and test case design
5. Defects, severity and priority
6. Confirmation, regression, smoke and sanity testing
7. Functional, non-functional, positive and negative testing
8. Black-box, white-box, grey-box and static/dynamic testing
9. Test design techniques
10. Testing levels, SIT and UAT
11. Agile and Scrum
12. Jira, Xray and Confluence
13. End-to-end QA workflow
14. Mini project and sample test cases
15. Interview revision checklist and practice questions

1. Software testing fundamentals

Software testing is a set of activities performed to evaluate the quality of a software product and determine whether it satisfies specified requirements. Testing helps detect defects, assess risk, and provide information for release and quality decisions.

Why testing exists

- Check whether specified requirements have been implemented.
- Discover defects before they cause customer, business, financial, safety or compliance problems.
- Validate that the product supports real user needs and workflows.
- Provide evidence about quality and residual risk.
- Check that changes have not unintentionally damaged existing behavior.

Testing does not prove that software is defect-free. It reduces uncertainty and risk by examining selected conditions and evidence.

Running example used throughout

Assume a course-management application. A logged-in student can view an available course and enroll once. If the student is already enrolled, the application must prevent a duplicate enrollment.

```
Requirement REQ-101
A logged-in student can enroll in an available course.
A student cannot enroll in the same course twice.

Possible tests:
• Available course + not yet enrolled → enrollment succeeds
• Already enrolled → duplicate enrollment is prevented
• Course unavailable → enrollment is rejected
• Not logged in → access is denied or login is required
```

2. QA, QC, verification and validation

Quality Assurance (QA)

A process-oriented set of activities intended to provide confidence that appropriate processes are established and followed so that quality requirements can be met. Examples include defining testing standards, reviewing processes, improving workflows and conducting process audits.

Quality Control (QC)

A product-oriented set of activities used to evaluate whether a work product meets specified quality requirements. Software testing is a major QC activity.

Verification

Evaluation of a work product to determine whether it satisfies specified requirements. The question is: “Are we building the product correctly?” Examples include requirement reviews, design reviews and code reviews.

Validation

Evaluation of a system to determine whether it meets user needs and its intended use. The question is: “Are we building the right product?” Executing an end-to-end workflow with realistic business data can contribute to validation.

Concept	Main focus	Example
QA	Process and defect prevention	Define a review process and testing standards
QC	Product quality and defect detection	Execute a test case and record the result
Verification	Conformance to specified requirements	Review a requirement or inspect code
Validation	Fitness for intended use	Business user completes a real workflow

3. SDLC and STLC

SDLC — Software Development Life Cycle

The overall set of activities used to specify, design, develop, test, deploy and maintain a software system.

```
Requirements → Design → Development → Testing → Deployment → Maintenance
      ■ QA/test activities can contribute throughout the lifecycle ■
```

SDLC phases

Requirements analysis

Understand business needs, users, constraints, scope and acceptance criteria. QA looks for ambiguity, missing rules and testability.

Design

Define architecture, components, interfaces, data structures, APIs and security approach. QA studies designs and plans suitable tests.

Development

Implement the software. Developers commonly perform unit tests and code reviews. QA can prepare test cases and data in parallel.

Testing

Execute planned and exploratory tests, compare actual with expected results, investigate failures and report defects.

Deployment

Release software to a target environment after required approvals and release criteria are met.

Maintenance

Correct defects, improve features, adapt to changing environments and test changes to existing software.

STLC — Software Testing Life Cycle

The testing-focused set of activities used to analyze test basis, plan testing, design and implement tests, prepare the environment, execute tests, manage defects and complete testing.

```
Requirement analysis
        ↓
Test planning
        ↓
Test analysis & design
        ↓
Test implementation / environment setup
        ↓
Test execution
        ↓
Defect handling + confirmation + regression
        ↓
Test completion / closure
```

SDLC	STLC
Covers the full software lifecycle	Focuses on testing activities
Includes development, deployment and maintenance	Includes test analysis, planning, design, execution and closure
Involves all relevant delivery roles	Primarily focuses on test work, while collaborating with the wider team

STLC phases in practice

1. Requirement analysis

Identify testable requirements, risks, dependencies, data needs and unanswered questions.

2. Test planning

Define scope, approach, resources, tools, environments, schedule, risks, entry criteria and exit criteria.

3. Test analysis and design

Derive test conditions, scenarios, test cases, expected results and coverage.

4. Test implementation

Organize cases, prepare test data, build suites, and prepare manual or automated tests.

5. Environment setup

Prepare the application build, accounts, permissions, integrations, data and configuration needed for execution.

6. Test execution

Run tests, record actual results, mark pass/fail/blocked, capture evidence and report defects.

7. Test completion

Summarize coverage, results, unresolved defects, risks, lessons learned and release-readiness information.

4. Test artifacts and test case design

Requirement

A documented business, user, system or technical need that the solution is expected to satisfy. Requirements may be recorded in user stories, specifications, tickets or other controlled documents.

Test scenario / test condition

A high-level situation, behavior or condition to be tested. “Verify course enrollment” is a scenario. A scenario can be expanded into multiple detailed test cases.

Test case

A documented set of preconditions, inputs/test data, execution steps and expected results designed to verify a particular objective or test condition.

Test data

Data required to execute a test, including valid, invalid, boundary, missing, duplicate or role-specific values.

Precondition

A condition that must be true before a test can be executed as designed. Example: the student is logged in and the course is available.

Expected result vs actual result

Expected result: the behavior defined by the requirement or test oracle. Actual result: the behavior observed during execution. A mismatch requires investigation; it is not automatically a confirmed software defect.

Test execution

The process of running a test, comparing observed behavior with expected results, and recording the outcome and evidence.

Test result statuses

Status	Meaning
Pass	Observed result satisfies the expected result.
Fail	Observed result does not satisfy the expected result; investigate and report if a product defect is confirmed.
Blocked	The test cannot be executed because a prerequisite or dependency is unavailable.
Not Run	The test has not yet been executed.

Sample test case

Field	Example
ID	TC-ENR-001
Title	Verify enrollment in an available course
Requirement	REQ-101
Preconditions	Student account exists; student is logged in; course is available; student is not enrolled.
Test data	Student: student01; Course: COURSE-101
Steps	1. Open course catalog. 2. Select COURSE-101. 3. Click Enroll. 4. Open enrollment details.
Expected result	Enrollment succeeds; course appears in the student's enrollment list; no duplicate record is created.
Actual result	Record what happened during execution.
Status	Set to Pass, Fail, Blocked or Not Run.

Test scenario vs test case

Scenario	Example test cases
Verify login	Valid credentials; invalid password; empty email; locked account; expired password.
Verify enrollment	Available course; unavailable course; duplicate enrollment; unauthenticated user.

5. Defects, severity and priority

Defect (bug)

A flaw in a work product or software implementation that can cause the system to fail to perform a required function or to behave in an unintended way. A test failure is evidence to investigate; environment or data problems may cause failures without a software defect.

Good defect report contents

- Clear title summarizing the problem
- Environment/build/browser or configuration
- Preconditions and test data
- Exact reproduction steps
- Expected result
- Actual result
- Reproducibility/frequency

- Screenshots, logs, API traces or other evidence
- Severity and priority, following team definitions

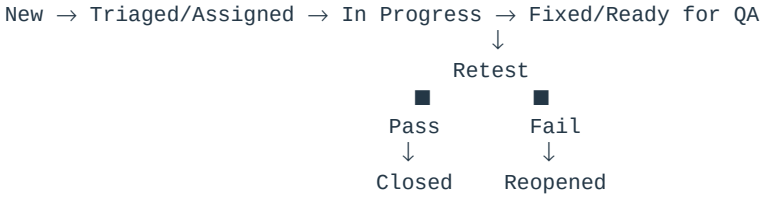
Severity vs priority

Severity describes the degree of impact a defect has on system operation, data, users or business processes. **Priority** describes the relative urgency with which the defect should be addressed. Exact scales are organization-specific.

Example	Possible severity	Possible priority	Reason
Payment processing is unavailable for all users	Critical/High	P1 / urgent	Major business function is blocked.
Typo in a highly visible campaign heading	Low	High	Technical impact is low, but business visibility may make it urgent.
Rare failure in a seldom-used internal report	High	Low/Medium	Impact may be serious but immediate urgency may be lower; context matters.

Do not treat these mappings as universal. Teams define scales and triage defects based on impact, frequency, users affected, workarounds, risk and release context.

Typical defect workflow



Statuses differ by organization and Jira workflow. A defect may also be rejected, duplicated, deferred or marked cannot reproduce after investigation.

6. Confirmation, regression, smoke and sanity testing

Confirmation testing / retesting

Testing after a defect fix to confirm that the original failure has been corrected. Usually reruns the failed test and any directly relevant checks.

Regression testing

Testing after a change to detect adverse effects on previously working functionality. The scope is selected based on change impact, dependencies and risk.

Smoke testing

A broad but relatively shallow set of checks used to assess whether a build is stable enough for more detailed testing.

Sanity testing

A focused check of a particular area or change to determine whether it behaves reasonably and whether further testing is worthwhile. Usage varies between organizations.

Activity	Core question	Example
Confirmation / retest	Did the specific defect get fixed?	The duplicate-enrollment test now rejects a second enrollment.
Regression	Did the change break something else?	Login, course history and enrollment still work.
Smoke	Is this build basically testable?	App opens, login works, dashboard loads.
Sanity	Does this specific change look correct?	Focused checks around the updated fee calculation.

7. Functional, non-functional, positive and negative testing

Functional testing

Evaluates whether software functions behave according to specified requirements and business rules. Examples: login, search, save, calculate, upload, enroll, authorize and generate a report.

Non-functional testing

Evaluates quality characteristics such as performance, security, usability, reliability, compatibility, accessibility and recoverability.

Type	Purpose	Example
Performance	Evaluate responsiveness/capacity under defined conditions	Measure API response time under workload.
Load	Evaluate behavior under expected/specified workload	Simulate typical peak traffic.
Stress	Evaluate behavior under excessive or stressful conditions	Increase traffic beyond expected capacity.
Security	Evaluate security controls and resistance to threats	Verify users cannot access another user's records.
Usability	Evaluate effectiveness and ease of use	Observe whether users can complete a workflow.
Compatibility	Evaluate behavior across specified environments	Test supported browsers and operating systems.
Reliability	Evaluate failure-free operation over time/conditions	Run a service continuously and monitor failures.
Accessibility	Evaluate usability by people with disabilities	Keyboard and screen-reader checks.
Recovery	Evaluate recovery after failures	Verify service/data recovery after interruption.

Positive testing

Tests system behavior with valid inputs and conditions. Example: register using a valid email and compliant password.

Negative testing

Tests system behavior with invalid inputs or prohibited conditions. Example: reject malformed email or unauthorized access. A negative test passes if the system rejects or safely handles the invalid condition as required.

8. Black-box, white-box, grey-box and static/dynamic testing

Black-box testing

Derives tests from specified behavior and requirements without requiring knowledge of internal implementation. Common techniques include equivalence partitioning, boundary value analysis, decision tables and state transitions.

White-box testing

Derives tests from internal structure or implementation. Common techniques include statement, branch, condition and data-flow testing.

Grey-box testing

Uses partial knowledge of internal design or architecture while evaluating behavior, often through external interfaces. Example: test an API and then verify persisted data.

Static testing

Evaluates a work product without executing the software. Examples include reviews and static code analysis.

Dynamic testing

Executes software and evaluates its runtime behavior. Examples include UI, API and runtime functional tests.

```
Two independent questions:
1. Is software executed?            Static vs Dynamic
2. What knowledge designs the test?  Black-box vs White-box (or grey-box)
```

Example: a Selenium UI test is usually dynamic and often black-box. But Selenium itself does not determine the classification; the test's design and information sources do.

9. Test design techniques

Equivalence partitioning

Divides input/output domains into partitions whose members are expected to be processed similarly, then selects representative values.

Boundary value analysis (BVA)

Derives tests from the boundaries of equivalence partitions, because defects often occur at or near range edges.

Example: allowed password length is 8–20 characters

Technique	Sample values	What it targets
Equivalence partitioning	7 (invalid), 12 (valid), 21 (invalid)	Representative values from valid/invalid groups.
Boundary value analysis	7, 8, 9, 19, 20, 21	Values at and near the lower and upper limits.

Decision table testing

Systematically tests combinations of conditions and their corresponding actions. Useful when business rules depend on multiple conditions.

Rule	Authenticated?	Permission?	Expected outcome
R1	Yes	Yes	Allow access

Rule	Authenticated?	Permission?	Expected outcome
R2	Yes	No	Deny access
R3	No	Yes	Deny access
R4	No	No	Deny access

State transition testing

Derives tests from states, events and transitions. Useful when the response depends on the current state, such as an account becoming locked after a specified number of failed attempts.

Use case testing

Derives tests from actor-system interactions designed to achieve a goal. It covers the main success path and relevant alternative/exception flows.

Experience-based techniques

Exploratory testing: test design, execution and learning happen together. Error guessing uses experience to anticipate likely defects. Checklist-based testing uses a list of items to be verified.

10. Testing levels, SIT and UAT

Unit testing

Tests individual units or components, typically in isolation. Example: verify a discount-calculation method.

Integration testing

Tests interactions between integrated components or systems. Example: verify that a service correctly persists a record through a repository and database.

System testing

Tests an integrated system against specified system requirements. Example: test the full login-to-enrollment workflow.

System Integration Testing (SIT)

Tests interfaces and interactions between integrated systems, subsystems or external services. Terminology varies: SIT is often treated as a broader integration phase.

Acceptance testing

Evaluates whether the system meets acceptance criteria, business needs or intended-use requirements.

User Acceptance Testing (UAT)

Acceptance testing performed by or with intended users, business representatives or customers to decide whether the system is acceptable for its intended use.

```
Typical conceptual progression (not a universal mandatory sequence):
Unit → Integration → System → System integration (where applicable) → Acceptance/UAT
```

Do not confuse test level with test type. For example, security testing can occur at multiple levels; API testing may be integration or system testing depending on scope.

11. Agile and Scrum

Agile

An approach to software development that emphasizes delivering valuable software incrementally, collaboration, feedback, and adapting to change. Agile is not one single process; Scrum is one framework used by Agile teams.

Waterfall vs Agile

Waterfall-style lifecycle	Agile/incremental approach
Phases are commonly planned and executed more sequentially.	Work is delivered in smaller increments with frequent feedback.
Late requirement changes may be expensive.	Changes can be incorporated through backlog refinement and reprioritization.
Testing may be concentrated in a later phase, depending on implementation.	Testing and quality activities occur throughout iterations.

Scrum accountabilities

Accountability	Core responsibility
Product Owner	Maximizes product value and manages/prioritizes the Product Backlog.
Scrum Master	Helps establish Scrum, supports effectiveness and addresses impediments.
Developers	Create a usable Increment each Sprint; the team includes the skills needed to deliver it.

Core Scrum terms and events

Term	Meaning
Product Backlog	Ordered, evolving list of work needed to improve the product.
Sprint	Fixed-length event of one month or less during which a usable Increment is created.
Sprint Planning	Team establishes why the Sprint is valuable, what can be done and how the work will be accomplished.
Sprint Backlog	Sprint Goal, selected Product Backlog items and the plan for delivering them.
Daily Scrum	Short daily event for Developers to inspect progress toward the Sprint Goal and adapt the plan.
Sprint Review	Inspect the Increment and discuss outcomes and future adaptations with stakeholders.
Sprint Retrospective	Inspect how the last Sprint went and identify improvements in quality and effectiveness.
User story	A common way to express a need from a user's perspective; not a mandatory Scrum artifact.
Epic	A large body of work often broken into smaller stories; tool/organization terminology varies.
Acceptance criteria	Conditions a work item must satisfy; usually defined by the team/product context.
Definition of Done	Formal shared description of the state of the Increment when it meets the required quality measures.

Typical user story format

> *As a **student**, I want to **enroll in an available course**, so that I can **start learning it**.*

Example acceptance criteria

```
Given the student is logged in and not already enrolled
When the student selects an available course and clicks Enroll
Then the enrollment is created and confirmation is shown

Given the student is already enrolled
When the student tries to enroll again
Then duplicate enrollment is prevented
```

12. Jira, Xray and Confluence

Jira

Jira is a work and issue-tracking platform used to manage stories, tasks, bugs, workflows, ownership and progress. The exact hierarchy and statuses depend on project configuration.

Typical Jira issue examples

```
Epic: Course Management
  Story: Student can enroll in a course
    Sub-task: Implement enrollment API
    Sub-task: Validate duplicate enrollment
  Bug: Duplicate enrollment is allowed
```

Xray

Xray is a test-management solution integrated with Jira. It can manage test cases, test plans, test sets, test executions, results and links between requirements and tests. Capabilities and issue types depend on the Xray version and configuration.

Key Xray concepts

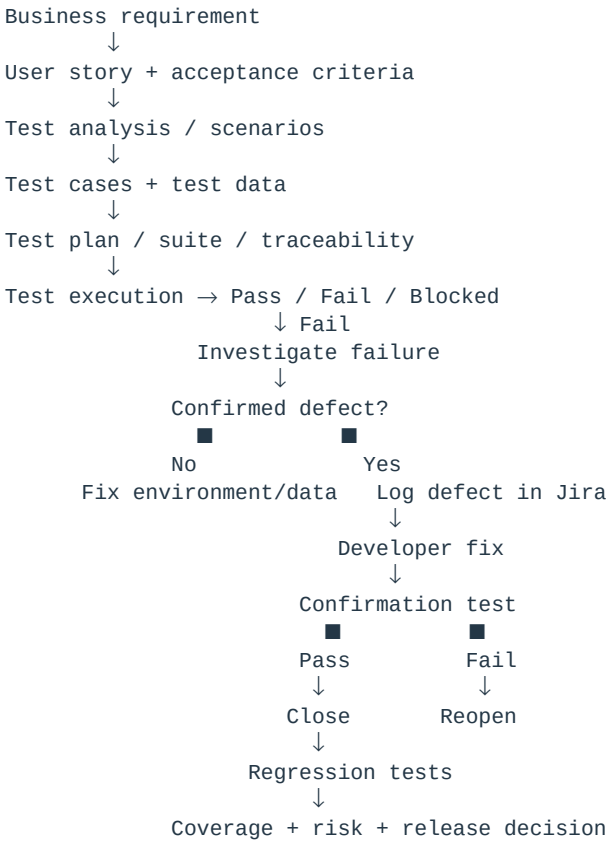
Concept	Purpose
Test	Defines what to test, often with steps and expected results.
Test Set	Groups tests for reuse or organization.
Test Plan	Organizes the tests intended for a testing scope/cycle.
Test Execution	Records the execution of tests and their results for a run.
Traceability	Links requirements/stories to tests, executions and defects where configured.

Confluence

Confluence is a team knowledge and documentation platform. It is commonly used for test strategies, requirement notes, technical documentation, meeting notes, release information and process guidance.

Tool	Think of it as
Jira	Work and issue tracking
Xray	Test management and traceability
Confluence	Documentation and team knowledge

13. End-to-end QA workflow



Traceability

Traceability is the ability to establish and follow relationships between related artifacts, such as requirements, test cases, test results and defects. It helps show what has been tested, which requirements remain uncovered, and which tests are affected by a change.

Release decisions

A tester provides evidence about coverage, test results, unresolved defects and risk. Release authorization belongs to the designated decision-makers and follows the organization's release criteria; a tester should not claim that a product is defect-free.

14. Mini project: test a course-enrollment feature

Requirement

A logged-in student can enroll in an available course. The system must prevent duplicate enrollment and reject enrollment when a course is unavailable.

Scenario list

- SC-01: Enroll in an available course.
- SC-02: Prevent duplicate enrollment.
- SC-03: Reject enrollment in an unavailable course.
- SC-04: Require authentication.
- SC-05: Verify enrollment confirmation and enrollment history.

- SC-06: Handle repeated submission or network delay without creating duplicate records.

Sample test cases

ID	Condition / steps	Expected result
TC-01	Logged in, available course, not enrolled; click Enroll.	Enrollment succeeds and appears in history.
TC-02	Already enrolled; click Enroll again.	No duplicate enrollment is created; appropriate message/behavior appears.
TC-03	Course is unavailable; attempt enrollment.	Enrollment is rejected according to requirements.
TC-04	Logged out; access enrollment action.	User is redirected to login or access is denied.
TC-05	Submit enrollment twice quickly.	System preserves data integrity and creates no duplicate record.

Defect report example

```
Title: Duplicate enrollment is created when Enroll is submitted twice

Environment: QA, build 1.4.2, Chrome (version recorded at execution)
Preconditions: Student is logged in and already enrolled in COURSE-101
Steps:
1. Open COURSE-101.
2. Click Enroll again.
3. Open enrollment history.

Expected: The system prevents duplicate enrollment.
Actual: A second enrollment record appears.
Evidence: Attach screenshot, request/response or relevant logs.
Severity/Priority: Assign according to team triage criteria.
```

Automation suitability

Stable, repeatable, high-value checks such as valid enrollment, duplicate prevention and access control may be good automation candidates. Exploratory investigation, one-off checks or tests requiring complex human judgment may remain manual. Automation should be selected based on risk, repeatability, stability, cost and expected maintenance.

15. Revision checklist and practice questions

Before moving to automation, you should be able to do these without looking at the guide:

- Explain SDLC vs STLC and name their typical activities.
- Write one scenario and at least five test cases from a short requirement.
- Distinguish expected result, actual result, test failure and confirmed defect.
- Explain severity vs priority using two realistic examples.
- Distinguish retesting, regression, smoke and sanity testing.
- Explain functional vs non-functional testing and positive vs negative testing.
- Classify black-box, white-box, grey-box, static and dynamic testing.
- Apply equivalence partitioning and boundary value analysis to a numeric or length constraint.
- Build a small decision table for two or three conditions.
- Explain unit, integration, system, SIT, acceptance and UAT.
- Explain Product Backlog, Sprint Backlog, Sprint, Daily Scrum, Review, Retrospective and Definition of Done.
- Describe what Jira, Xray and Confluence are used for.

- Walk through requirement → test → defect → fix → retest → regression.

Practice questions

1. A test fails because the QA database is unavailable. Is this automatically a software defect? Explain.
2. A password must be 8–20 characters. Design equivalence partitions and boundary tests.
3. Explain why 100% statement coverage does not guarantee defect-free software.
4. A bug fix passes its original test. What else might QA do and why?
5. Explain the difference between system testing and SIT using an example.
6. What evidence belongs in a high-quality defect report?
7. When might an API test be considered integration testing rather than system testing?
8. Why should not every test be automated through Selenium?
9. Explain how a requirement can be traced to test results and defects.
10. Describe the difference between a Scrum Sprint Review and Sprint Retrospective.

Suggested references

For standardized terminology, consult the ISTQB Certified Tester Foundation Level syllabus and official glossary: istqb.org and glossary.istqb.org. For tool-specific workflow details, consult official Jira and Xray documentation. Organizations may customize workflows and use some testing terms differently.

End of Phase 1 Learning Guide